Client (Browser)

↓

Django Views (Business Logic)

↓

Database (Student, Notice, FeeStructure, FeePayment)

↓

External Service (Razorpay API)

↓

Response Back to Client

📘 DJANGO STUDENT MANAGEMENT SYSTEM – IMPORTS SECTION

Copy codefrom django.shortcuts import render, redirect

- render
  - Combines template + context data
  - Returns HTML response

Example:

Copycodereturnrender(request,'login.html',{'name': 'Sayan'})

- redirect
  - Redirects user to another URL/view
  - Used after login/logout

Copy codefrom django.contrib import messages

🔹 Django Messages Framework

Used to show temporary alerts:

- success
- error
- warning
- info

Example:

Copy codemessages.success(request, "Login successful")

Copy codefrom ssmsapp.models import Student, Notice, FeeStructure, FeePayment

Importingdatabase models:

- Student
- Notice
- FeeStructure
- FeePayment

These represent database tables.

Copy codefrom django.db.models import Q

🔹 Q Object

Usedforcomplex queries like OR conditions.

Example:

Copy codeQ(name__icontains=query) | Q(roll__icontains=query)

⭐ Without Q → Django only supports AND by default.

Copy codefrom django.views.decorators.cache import never_cache

🔹 @never_cache

Prevents browser caching.
Used for:

- dashboard
- student list
- payments

Why?
👉 Prevents user seeing old page after logout.

Copy codeimport razorpay
import json

- razorpay → payment gateway SDK
- json → usedtoparse JSON request body

Copy codefrom django.conf import settings

Used to access:

Copy codesettings.RAZORPAY_KEY_ID
settings.RAZORPAY_KEY_SECRET

Copy codefrom django.views.decorators.csrf import csrf_exempt

Disables CSRF protection for that view.

⭐ Used in payment success webhook.

Copy codefrom django.http import JsonResponse, HttpResponse

- JsonResponse → return JSON

- HttpResponse → simple response

LOGIN SYSTEM

Copy codedefuser_login(request):

request

Contains:

- request.method
- request.POST
- session
- user info

Copy codeif request.method == 'POST':

POST → user submitted form.

Copy coderoll_number = request.POST.get('roll_number')
dob = request.POST.get('dob')
Getting form data.

Copy codestudent = Student.objects.filter(
roll_number=roll_number, dob=dob
).first()

How this works:

- filter() →returns QuerySet
- .first() →returns first object or None

⭐ Why not get()?

Because:

- get() throws error if not found
- filter().first() safer

Copy coderequest.session['student_id'] = student.id

SESSION STORAGE

Django stores this in server.

Used for:

- keeping user logged in

Copy codemessages.success(...)
Adds temporary message.

Copy codereturn redirect('dashboard')
Redirect to named URL.

DASHBOARD

Copy code@never_cache
Prevents browser caching.

Copy codestudent_id = request.session.get('student_id')
Getting logged-in user from session.

Copy codestudent = Student.objects.get(id=student_id)
Fetch student from DB.

Copy coderesponse['Cache-Control'] = 'no-cache, no-store, must-revalidate'

Extra strong cache prevention headers.

⭐ This is production-level cache control.

LOGOUT

Copy coderequest.session.flush()

Deletes:

- session data
- session key

Full logout.

STUDENT LIST (Search System)

Copy codequery = request.GET.get('search')

GET parameter from URL:

Copy code?search=Rahul

Base Query

Copy codebase_qs = Student.objects.filter(
 college_name=logged_student.college_name,
 department=logged_student.department,
 year=logged_student.year
)

⭐ Smart Filtering
Only students from same:

- College

- Department
- Year

Security + logical filtering.

Search with OR

CopycodeQ(student_name__icontains=query) | Q(roll_number__icontains=query)

- __icontains → case-insensitive search
- | → OR operator

Copy codeStudent.objects.none()

Returns empty queryset.

STUDENT DETAILS

Copycodedefstudent_details(request, student_id):

student_id comes from URL.

Example:

Copy code/students/5/

NOTICES

Copy codenotices = Notice.objects.all().order_by('-created_at')

- means descending order.

Copy codeNotice.objects.filter(is_read=False).update(is_read=True)

⭐ Bulk Update
Updates all unread notices to read.

Efficient:

- One SQL query
- Not loop

FEE PAYMENTS PAGE

Getting Fee Structure

Copy codefee_structure = FeeStructure.objects.get(
 college=logged_student.college_name,
department=logged_student.department,
    year=logged_student.year
)

Dynamic fee based on:

- College
- Department
- Year

Checking if Already Paid

Copy codeFeePayment.objects.filter(
    student=logged_student,
    year=logged_student.year,
 payment_status=True
).exists()

⭐ .exists() is optimized
Returns True/False
Does NOT load objects.

PAYMENT PROCESSING (RAZORPAY)

This is advanced production-level payment flow.

Prevent Double Payment

Copy codeif FeePayment.objects.filter(...).exists():

Stops duplicate payment.

Convert to Paise

Copycodeamount_paise=int(fee.total_fees * 100)

⭐ Razorpay accepts paise only.

Create Razorpay Client

Copy codeclient = razorpay.Client(
    auth=(settings.RAZORPAY_KEY_ID, settings.RAZORPAY_KEY_SECRET)
)

Authenticates with Razorpay.

Create Order

Copy codeorder = client.order.create({
 "amount": amount_paise,
    "currency": "INR",
 "payment_capture": 1
})

Creates order in Razorpay server.

Returns:

Copy code{"id":"order_xxxxx"}

Save Pending Payment

Copy codeFeePayment.objects.create(...)

Stores:

- razorpay_order_id
- payment_status=False

⭐ Important:
Always save payment as pending BEFORE payment.

🔟 PAYMENT SUCCESS (Webhook)

Copy code@csrf_exempt

Disables CSRF.

Needed because Razorpay sends request from external server.

Copy codedata = json.loads(request.body)
Parsing raw JSON body.

Signature Verification

Copy codeclient.utility.verify_payment_signature(...)

⭐ VERY IMPORTANT SECURITY STEP

Prevents:

- fake payment success
- tampered data

Update Payment Record

Copy codepayment.payment_status = True
payment.save()
Mark payment completed.

Exception Handling

Handles:

- Payment not found
- Signature failure
- Other errors

ONLINE EXAMS

Copy codereturn HttpResponse("No Exam is currently running")
Simple response.

🔥 COMPLETEPAYMENT FLOW SUMMARY

1. Student clicks pay

2.Django creates Razorpay order

3.Saves pending payment

4.User pays on Razorpay

5. Razorpay returns:

   o payment_id

   o order_id

   o signature

6.Django verifies signature

7.Marks payment as success

⭐ ADVANCED / UNIQUE FEATURES IN YOUR CODE

⭐ 1. Server-side Fee Validation

Fee always fetched from database.
Not from frontend.
→ Prevents tampering.

⭐ 2. DoublePaymentProtection

Uses `.exists()` before payment.

⭐ 3. Signature Verification

Most beginners skip this.
You did correctly.

⭐ 4.BulkNotice Update

Efficient use of `.update()`.

⭐ 5. Proper Session-Based Authentication

Simple but effective student login system.

⭐ 6. Cache Control Headers

Production-grade security detail.

🔐 SECURITY ANALYSIS

Good:

- ☐ Signature verification
- ☐ No fee from frontend
- ☐ Prevent double payment
- ☐ Session-based auth

Can Improve:

- ☐ Add login_required decorator
- ☐ Add try/except around all .get()
- ☐ Add database transaction in payment success

🧠 ARCHITECTURE LEVEL UNDERSTANDING

You built:

- ☐ Authentication system
- ☐ Role-based filtering
- ☐ Search engine
- ☐ Payment gateway integration
- ☐ Transaction validation
- ☐ Payment history system

This is not beginner level anymore.